Reviewer Pack — Minimal Rank of Geometric Quantization over Manifolds vs. Re…

Entry 32ff8743153e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 22:54:50 UTC

Minimal Rank of Geometric Quantization over Manifolds vs. Resolution Proof Length for Non-clausal Tseitin Formulas

Entry ID: `32ff8743153e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 22:54:50 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Non-clausal Tseitin Formulas

Statement:

> ['For every non-clausal Tseitin formula with n variables, the minimal rank of its geometric quantization over a symplectic manifold is upper bounded by a function f(n) where E[f(n)] = Θ(n log(n)) and E[resolution_proof_length] = Θ(f(n)).', 'For any counterexample to this conjecture, there exists an instance where resolution proof length grows superpolynomially with the number of variables.']

Rationale (proposer's reasoning):

> ['Geometric quantization provides a bridge between symplectic geometry and quantum theory. This conjecture suggests that the complexity of resolving non-clausal Tseitin formulas might be related to geometric invariants, potentially revealing new insights into computational complexity.', 'This relationship has not been extensively explored before, making it a novel application of geometric quantization in complexity theory.']

Taxonomy category: `Geometric_Quantization_vs_Resolution_Proof` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `31df5bf503abfa34`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40, the average minimal rank of geometric quantization over symplectic manifolds is within Θ(n log(n)) and the average resolution proof length is within Θ(minimal rank). The conjecture is falsified if any n ≤ 40 exhibits a resolution proof length that grows superpolynomially with n.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `title:(geometric quantization AND symplectic manifold) AND (resolution proof length OR non-clausal Tseitin formulas)` - `quantization rank AND (Tseitin formulas OR resolution complexity) AND asymptotic behavior` - `symplectic geometry AND algorithmic aspects IN resolution proof length AND geometric quantization`

Top relevant hits considered: - [http://arxiv.org/abs/2407.15508v3] Compensate Quantization Errors+: Quantized Models Are Inquisitive Learners - [http://arxiv.org/abs/1912.08047v1] The Benefits of Affine Quantization - [http://arxiv.org/abs/2406.16299v1] Compensate Quantization Errors: Make Weights Hierarchical to Compensate Each Other - [http://arxiv.org/abs/1612.04764v1] Cohomological aspects on complex and symplectic manifolds - [http://arxiv.org/abs/1811.09984v4] Translated points for contactomorphisms of prequantization spaces over monotone symplectic toric manifolds - [http://arxiv.org/abs/math/0601540v1] Symplectic forms and surfaces of negative square

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(n):
            clauses.append(f'{variables[i]}')
            clauses.append(f'-{variables[i]}')
        for i in range(n):
            for j in range(i+1, n):
                clauses.append(f'{-variables[i]} {variables[j]} -{variables[j]}')
                clauses.append(f'{variables[i]} {-variables[j]} {variables[j]}')
        return ' '.join(clauses)

    def resolution_proof_length(formula):
        # Simplified version of resolution proof length calculation
        # This is a placeholder and should be replaced with actual logic
```

```
        return len(formula.split())

    n = random.randint(5, 40)
    formula = generate_tseitin_formula(n)
    rank = n * math.log(n)  # Placeholder for minimal rank of geometric quantization
    proof_length = resolution_proof_length(formula)

    return {
        "metric_name": "Resolution Proof Length",
        "metric_value": proof_length,
        "instances_tested": 1,
        "conjecture_holds": proof_length <= rank,
        "counterexample": "" if proof_length <= rank else f"Proof length {proof_length} exceeds ex
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Proof length exceeds expected rank\" first_fail
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3bda23b8.py", line 56, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3bda23b8.py", line 39, in run_trial
    formula = generate_tseitin_formula(n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3bda23b8.py", line 29, in generate_tsei
    clauses.append(f'{-variables[i]} {variables[j]} -{variables[j]}')
                     ^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means we cannot verify the conjecture's conditions for n ≤ 40. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14423
2	preregistration	ollama_remote	glm4:latest	0	0	9373
3	novelty	ollama_remote	glm4:latest	0	0	8548
4	novelty	ollama_remote	glm4:latest	0	0	9681
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12907
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9717
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7073
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7246
9	judge	ollama_remote	glm4:latest	0	0	11556

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 90524 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/32ff8743153e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/32ff8743153e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/32ff8743153e.tar.gz` (if generated)